

ĐẠI HỌC CÔNG NGHIỆP HÀ NỘI
KHOA CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



BÁO CÁO THÍ NGHIỆM/THỰC NGHIỆM
HỌC PHẦN: TRÍ TUỆ NHÂN TẠO

ĐỀ TÀI: PHÁT TRIỂN HỆ THỐNG AI PHÂN LOẠI LỆNH BẰNG MẠNG NEURAL, ỨNG DỤNG VÀO NGÔI NHÀ THÔNG MINH (SMART HOME)

GVHD: Mai Thanh Hồng

Nhóm: 2

Sinh Viên Thực hiện:	Đậu Vũ Công	2023601096
	Trương Quang Hải	2023600664
	Đàm Minh Hiếu	2023604325
	Lê Đức Thắng	2023600455
	Hồ Huy Tiến	2023601149

Lớp: 20251IT6094001- K18

Hà Nội - 2025

PHIẾU HỌC TẬP CÁ NHÂN/NHÓM

I. Thông tin chung

Tên lớp: 20251IT6094001

Khóa: 18

Tên nhóm: Nhóm 02

Họ và tên thành viên trong nhóm: Đâu Vũ Công

Trương Quang Hải

Đàm Minh Hiếu

Lê Đức Thắng

Hồ Huy Tiến

II. Nội dung học tập

Tên chủ đề: Phát triển hệ thống AI phân loại lệnh bằng mạng neural, ứng dụng vào ngôi nhà thông minh.

Hoạt động của sinh viên: 1. Hoạt động/Nội dung 1: Tổ chức, thảo luận và phân chia nhiệm vụ cho các thành viên nhóm. Mục tiêu/chuẩn đầu ra: L4.

2. Hoạt động/Nội dung 2: Tìm hiểu khái niệm, định lý, cách giải quyết bằng mạng neural nhân tạo. Mục tiêu/chuẩn đầu ra: L3.

3. Hoạt động/Nội dung 3: Giải bài toán bằng MLP và thực hiện cài đặt thuật toán với ngôn ngữ lập trình Python. Mục tiêu/chuẩn đầu ra: L3.

Sản phẩm nghiên cứu: 1. Bản báo cáo thực nghiệm xây dựng mạng neural nhân tạo và sử dụng các công cụ dành cho Deep Learning và Machine Learning để giải quyết bài toán.

2. Sản phẩm đi kèm.

III. Nhiệm vụ học tập

1. Hoàn thành Tiểu luận, Bài tập lớn, Đồ án/Dự án theo đúng thời gian quy định.
2. Báo cáo sản phẩm nghiên cứu theo chủ đề được giao trước giảng viên và những sinh viên khác.

KẾ HOẠCH THỰC HIỆN BÁO CÁO THÍ NGHIỆM THỰC NGHIỆM

Tên lớp: 20251IT6094001

Khóa: K18

Tên nhóm: Nhóm 02

Tên chủ đề: Phát triển hệ thống AI phân loại lệnh bằng mạng neural, ứng dụng vào ngôi nhà thông minh

Tuần	Người thực hiện	Nội dung công việc	Kết quả đạt được	Phương pháp thực hiện
Tuần 1	Cả nhóm	Tổ chức, phân chia nhiệm vụ cho các thành viên nhóm.	Bầu ra nhóm trưởng và phân chia được nhiệm vụ cho các thành viên.	Trao đổi, thảo luận giữa các thành viên qua Zoom
	Trương Quang Hải	Lý do lựa chọn đề tài và mục tiêu đề tài	Xác định được đề tài và mục tiêu của đề tài	Tìm hiểu trên internet.
	Lê Đức Thắng	Tìm hiểu về các phương pháp nghiên cứu	Đã hiểu xác định được phương pháp nghiên cứu	Tìm hiểu trên internet.
Tuần 2	Cả nhóm	Tìm hiểu về ngôn ngữ Python và công cụ Visual Studio Code. Thực hiện cài đặt thuật toán bằng Python.	Đã hiểu thuật toán và thực hiện cài đặt thành công bài toán bằng ngôn ngữ Python.	Tìm hiểu tài liệu và cài đặt chương trình
	Cả nhóm	Tìm hiểu về Mạng neural	Nắm được quy tắc xây dựng mạng	Tìm hiểu trên internet
	Đàm Minh Hiếu	Ưu, nhược điểm của MLP	Đã tìm hiểu	Tìm hiểu trên internet

Tuần	Người thực hiện	Nội dung công việc	Kết quả đạt được	Phương pháp thực hiện
Tuần 3	Đậu Vũ Công	Tìm hiểu về tiền xử lý và trực quan hóa dữ liệu	Đã tìm hiểu	Tìm hiểu trên internet.
	Lê Đức Thắng	Demo chương trình	Đã chạy demo chương trình	Tìm hiểu trên internet.
	Nguyễn Văn Tiến	Làm powerpoint và viết hướng dẫn sử dụng chương trình	Đã viết được hướng dẫn sử dụng và hoàn thành powerpoint	Tìm hiểu trên internet.
Tuần 4	Cả nhóm	Tổng hợp những nội dung đã làm được và làm báo cáo thực nghiệm.	Hoàn thành báo cáo thực nghiệm.	Trao đổi, thảo luận giữa các thành viên. Thực hiện trên phần mềm Microsoft Word.
		Sửa lỗi, hoàn thiện báo cáo thực nghiệm và chương trình.	Hoàn thành.	Trao đổi, thảo luận giữa các thành viên.

Ngày tháng năm 2025

Giảng viên

**BỘ CÔNG THƯƠNG
TRƯỜNG ĐẠI HỌC CÔNG
NGHIỆP HÀ NỘI**

**CỘNG HÒA XÃ HỘI CHỦ NGHĨA
VIỆT NAM
Độc lập - Tự do - Hạnh phúc**

PHIẾU ĐÁNH GIÁ BÁO CÁO THÍ NGHIỆM/THỰC NGHIỆM

I. THÔNG TIN CHUNG

Người đánh giá: Mai Thanh Hồng

Khóa: 18

Đơn vị công tác: Khoa CNTT

Tên lớp: 20251IT6094001

Họ và tên thành viên trong nhóm:

Đậu Vũ Công

Trương Quang Hải

Đàm Minh Hiếu

Lê Đức Thắng

Nguyễn Văn Tiến

Tên nhóm: Nhóm 02

Tên chủ đề: Phát triển hệ thống AI phân loại lệnh bằng mạng neural, ứng dụng vào ngôi nhà thông minh.

II. ĐÁNH GIÁ

T T	Mục tiêu/chuẩn đầu ra học phần	Tiêu chí đánh giá sản phẩm	Điểm tối đa	Điểm đánh giá
1	L3	Phân tích được các bước thực hiện một công nghệ, kỹ thuật TTNT và áp dụng công cụ vào triển khai một thực nghiệm.	7	
2	L4	Chủ động tham gia, tổ chức các hoạt động và phát triển nhóm.	3	
Tổng			10	

Ngày tháng năm 2025

MỤC LỤC

MỤC LỤC	7
DANH MỤC HÌNH ẢNH	9
CHƯƠNG 1: GIỚI THIỆU	10
1.1. Lý do chọn đề tài	10
1.2. Mục Tiêu của đề tài	11
1.3. Nhiệm vụ	12
1.4. Tổng quan bài toán phân loại lệnh (smart home)	13
1.4.1. Các loại lệnh trong Nhà thông minh (Smart Home)	13
1.4.2. Đặc điểm và Tiêu chuẩn của dữ liệu lệnh tiếng Việt	14
1.5. Giới thiệu mạng neural & Hướng giải quyết	16
1.5.1. Mạng neural	16
1.5.2. Hướng giải quyết	16
CHƯƠNG 2: PHÂN LOẠI LỆNH BẰNG MẠNG NEURAL NHÂN TẠO (MLP)	19
2.1. Công nghệ và thư viện sử dụng	19
2.2. Kỹ thuật sử dụng	21
2.2.1. Tổng quan về MLP	22
2.2.2. Lý do lựa chọn kỹ thuật MLP	23
2.2.3. Cấu trúc mô hình MLP trong đề tài	23
2.2.4. Hàm mất mát và phương pháp tối ưu	23
2.2.5. Cơ chế hoạt động của kỹ thuật MLP	24
2.2.6. Ưu điểm và hạn chế	24
2.3. Quy trình thực hiện	25
2.3.1. Chuẩn bị dữ liệu (Dataset)	26
2.3.2. Xử lý dữ liệu (Preprocessing)	28
2.3.3. Xây dựng mô hình (Model)	31
2.3.4. Dự đoán và đánh giá	33

2.3.5. Xây dựng Mobile App và giả lập phần cứng	34
2.3.6. Tối ưu mô hình	36
2.4. Kết luận.....	36
CHƯƠNG 3: KẾT QUẢ	37
3.1. Kết quả đạt được.....	38
3.1.1. Video Demo & source code	38
3.1.2. Kết quả về mặt ứng dụng.....	39
3.1.3. Hạn chế của đề tài.....	41
3.2. Hướng phát triển trong tương lai	43
3.2.1. Mở rộng và đa dạng hóa dữ liệu.....	43
3.2.2. Tối ưu mô hình MLP	44
3.2.3. Nghiên cứu các mô hình nâng cao	44
3.2.4. Triển khai thực tế trên thiết bị IoT	45
3.2.5. Phát triển giao diện người dùng	45
3.2.6. Phát triển hệ thống đa phương thức.....	46
3.3. Kết luận chung.....	46
TÀI LIỆU THAM KHẢO	48

DANH MỤC HÌNH ẢNH

Hình 1: Sơ đồ luồng hoạt động của dự án	25
Hình 2: Cấu trúc thư mục dự án	26
Hình 3: Dữ liệu dạng bảng 2 cột (text - intent)..	28
Hình 4: Biểu đồ trực quan hóa dữ liệu	31
Hình 5: Minh họa mạng neural MLP	33
Hình 6: Code đánh giá mô hình.....	34
Hình 7: Giao diện mobile app	35

CHƯƠNG 1: GIỚI THIỆU

1.1. Lý do chọn đề tài

Trong bối cảnh cuộc Cách mạng Công nghiệp 4.0 đang diễn ra mạnh mẽ trên toàn cầu, khái niệm "Nhà thông minh" (Smart Home) đã vượt qua ranh giới của những bộ phim viễn tưởng để trở thành một tiêu chuẩn sống mới trong xã hội hiện đại. Sự kết hợp giữa Internet vạn vật (IoT), Điện toán đám mây (Cloud Computing) và Trí tuệ nhân tạo (AI) đang định hình lại cách con người tương tác với không gian sống. Tuy nhiên, cốt lõi của một ngôi nhà thực sự thông minh không chỉ nằm ở việc trang bị hàng loạt các thiết bị đắt tiền hay cảm biến hiện đại, mà nằm ở khả năng "thấu hiểu" và tương tác hai chiều với người sử dụng.

Thực tế hiện nay cho thấy, rào cản lớn nhất trong việc phổ cập Smart Home tại Việt Nam chính là giao diện điều khiển. Các phương thức truyền thống như công tắc vật lý thì thiếu linh hoạt, trong khi giao diện chạm (Touch Interface) trên Smartphone lại đòi hỏi người dùng phải trải qua nhiều bước thao tác (mở khóa, tìm ứng dụng, chọn menu). Điều này đi ngược lại với nguyên lý "tiện nghi tối đa" và gây khó khăn lớn cho những đối tượng đặc thù như người cao tuổi, trẻ em hoặc người khiếm thị.

Để giải quyết vấn đề này, hệ thống cần một module xử lý trung tâm đóng vai trò như một "bộ não ngôn ngữ", có khả năng chuyển đổi những câu lệnh tự nhiên, đòi hỏi thường của con người thành các tín hiệu kỹ thuật số chính xác (Digital Signals) mà thiết bị IoT có thể thực thi. Đây chính là bài toán giao tiếp Người - Máy (Human-Computer Interaction) thông qua giao diện giọng nói và văn bản.

Nội dung chương này tập trung đi sâu nghiên cứu về cơ chế giao tiếp và xử lý thông tin trong hệ thống nhà thông minh, cụ thể bao gồm các khía cạnh:

- **Tìm hiểu về cấu trúc lệnh điều khiển:** Phân tích sâu sắc các đặc điểm ngôn ngữ học, cấu trúc cú pháp (Syntax) và ngữ nghĩa (Semantics) của các câu lệnh tiếng Việt. Khác với tiếng Anh, tiếng Việt có tính đơn lập và phụ thuộc ngữ cảnh cao, đòi hỏi các phương pháp xử lý đặc thù.
- **Phát biểu bài toán phân loại ý định (Intent Classification):** Xác định rõ vấn đề cốt lõi không phải là nhận dạng từng từ khóa riêng lẻ, mà là hiểu được

"ý định" (Intent) ẩn sau câu nói. Làm sao để máy tính phân biệt được sự khác nhau giữa câu trần thuật "Trời tối quá" và câu mệnh lệnh "Bật đèn lên", dù cả hai đều ám chỉ việc cần ánh sáng.

- **Nghiên cứu các phương pháp Xử lý ngôn ngữ tự nhiên (NLP):** Khảo sát sự chuyển dịch từ các phương pháp thống kê truyền thống (như TF-IDF) sang các mô hình Học sâu (Deep Learning). Trọng tâm là tìm hiểu quy trình xây dựng và huấn luyện Mạng nơ-ron nhân tạo (Artificial Neural Network - ANN) để giải quyết bài toán phân lớp văn bản ngắn với độ chính xác cao và độ trễ thấp.

1.2. Mục Tiêu của đề tài

Đề tài hướng đến việc thiết kế và triển khai một hệ thống có khả năng nhận diện và phân loại chính xác các lệnh được nói ra bởi người dùng. Các mục tiêu cụ thể gồm:

- **Mục tiêu tổng quát:** Xây dựng một mô hình trí tuệ nhân tạo có thể xử lý tín hiệu âm thanh và phân loại các lệnh giọng nói thành những nhóm hành động cụ thể như `play_music`, `open_door`, `close_door`, ... nhằm hỗ trợ điều khiển các thiết bị trong hệ thống nhà thông minh.
- **Mục tiêu cụ thể:**
 - **Thu thập và xử lý dữ liệu âm thanh:** Tạo bộ dữ liệu gồm các câu lệnh mẫu, chuẩn hóa, loại nhiễu và trích xuất đặc trưng (MFCC hoặc Mel-Spectrogram).
 - **Xây dựng và huấn luyện mô hình mạng nơ-ron:** Lựa chọn kiến trúc (ANN/CNN/RNN hoặc kết hợp), tối ưu tham số để đạt độ chính xác cao trong phân loại lệnh.
 - **Xây dựng công cụ phân loại lệnh đầu vào thời gian thực:** Nhận âm thanh từ micro, xử lý và đưa ra kết quả phân loại ngay lập tức.
 - **Đánh giá hiệu suất mô hình:** Sử dụng các chỉ số như accuracy, confusion matrix, F1-score để đánh giá mức độ chính xác và ổn định.
 - **Tích hợp mô hình vào hệ thống Smart Home:** Chuyển kết quả phân loại thành hành động tương ứng như mở cửa, đóng cửa, bật/tắt nhạc.

- **Đề xuất hướng phát triển:** Mở rộng bộ lệnh, cải thiện khả năng nhận diện trong môi trường có nhiễu, nhận dạng nhiều người dùng, hỗ trợ đa ngôn ngữ, ...
- **Kết quả mong đợi:** Xây dựng được mô hình phân loại lệnh hoạt động ổn định. Tỷ lệ nhận diện các lệnh phổ biến đạt mức chấp nhận được (ví dụ $\geq 85\%$ hoặc theo yêu cầu môn học).
- Hệ thống có thể áp dụng thực tế ở mức thử nghiệm và dễ dàng mở rộng trong tương lai.

1.3. Nhiệm vụ

Xuất phát từ những yêu cầu cấp thiết của thực tiễn và nền tảng lý thuyết nêu trên, để xây dựng thành công một hệ thống phân loại lệnh hoàn chỉnh và có tính ứng dụng cao, đề tài của nhóm em sẽ tập trung giải quyết 4 nhiệm vụ trọng tâm sau:

- **Nhiệm vụ 1:** Tìm hiểu khái quát và chuyên sâu về Xử lý ngôn ngữ **tự nhiên (NLP)** Nghiên cứu các thách thức đặc thù của tiếng Việt trong xử lý máy tính: vấn đề tách từ (Word Segmentation), xử lý từ đồng nghĩa, đa nghĩa và các hư từ (Stopwords).
Tìm hiểu về kiến trúc Mạng nơ-ron truyền thẳng (Feed Forward Neural Network), cơ chế lan truyền ngược (Backpropagation) và các hàm kích hoạt (Activation Function) giúp mô hình học được các đặc trưng phi tuyến tính của dữ liệu.
- **Nhiệm vụ 2:** Thu thập và xây dựng bộ dữ liệu (**Dataset**) **chuẩn hóa**. Dữ liệu là yếu tố sống còn của các mô hình học máy. Nhóm sẽ tiến hành thu thập, tổng hợp các mẫu câu lệnh điều khiển nhà thông minh từ thực tế sử dụng, đảm bảo tính đa dạng về văn phong (bao gồm cả tiếng phổ thông, từ địa phương, câu rút gọn và tiếng lóng). Thực hiện quy trình gán nhãn dữ liệu (Data Labeling) một cách thủ công và chính xác theo các nhóm ý định (Intent) đã định nghĩa, ví dụ: **turn_on**, **turn_off**, **check_status**, **play_music**. Đây sẽ là tập dữ liệu nền tảng (Ground Truth) để huấn luyện mô hình.
- **Nhiệm vụ 3:** Xây dựng quy trình xử lý (Pipeline) cho **bài toán phân loại** Đây là nhiệm vụ kỹ thuật cốt lõi, bao gồm 3 công đoạn liên tiếp:

- **Tiền xử lý dữ liệu (Preprocessing):** Xây dựng module làm sạch dữ liệu đầu vào, bao gồm chuẩn hóa văn bản, loại bỏ ký tự nhiễu, xử lý các từ viết tắt và quan trọng nhất là tách từ tiếng Việt (Tokenization) để máy tính hiểu đúng đơn vị ngữ nghĩa.
- **Trích xuất đặc trưng (Feature Extraction):** Ứng dụng phương pháp (Bag of Words) để biến đổi các câu lệnh từ dạng chuỗi ký tự sang dạng vector số học (Vectorization). Đây là bước cầu nối giúp chuyển đổi ngôn ngữ tự nhiên sang ngôn ngữ toán học mà máy tính có thể tính toán được.
- **Phân loại ý định (Classification):** Thiết kế, huấn luyện và tinh chỉnh tham số (Hyperparameter tuning) cho mô hình AI để đạt được độ chính xác tối ưu trên tập kiểm thử.
- **Nhiệm vụ 4:** Cài đặt thử nghiệm, tích hợp hệ thống và đánh giá hiệu năng. Phát triển ứng dụng di động (Mobile App) đóng vai trò là Client, cung cấp giao diện tương tác thân thiện cho người dùng. Xây dựng hệ thống giả lập thiết bị phần cứng (IoT Simulator) sử dụng giao thức MQTT (Message Queuing Telemetry Transport). Mục tiêu là kiểm chứng khả năng hoạt động thời gian thực (Real-time) của mô hình, từ lúc người dùng ra lệnh đến lúc thiết bị phản hồi trạng thái.

1.4. Tổng quan bài toán phân loại lệnh (smart home)

1.4.1. Các loại lệnh trong Nhà thông minh (Smart Home)

Trong hệ sinh thái IoT tại Việt Nam, phương thức giao tiếp giữa người và vật đang có sự chuyển dịch rõ rệt. Nếu như giao diện chạm (Touch) mang lại sự trực quan, thì giao diện giọng nói/văn bản (Voice/Text) lại mang đến sự tự nhiên và tiện lợi. Trong phạm vi đề tài này, chúng tôi tập trung sâu vào việc phân tích và xử lý giao diện ngôn ngữ.

Một "Lệnh" (Command) trong ngữ cảnh SmartHome được định nghĩa chặt chẽ là một đơn vị thông tin đầu vào (Input) xuất phát từ ý chí của người dùng, mang theo mong muốn cụ thể nhằm thay đổi trạng thái hoạt động của thiết bị hoặc truy xuất thông tin từ hệ thống. Khác với các văn bản hành chính hay văn học mang tính

trần thuật, mô tả, dữ liệu "Lệnh" có đặc thù là rất ngắn gọn, súc tích, hàm lượng thông tin cao và mang tính mệnh lệnh trực tiếp.

Dựa trên chức năng, ngữ nghĩa và mục đích sử dụng, chúng tôi phân loại không gian các lệnh điều khiển thành 3 nhóm chính như sau:

- 1. Nhóm lệnh điều khiển trạng thái (Control Commands):** Đây là nhóm lệnh chiếm tỷ trọng lớn nhất (khoảng 70-80%) trong các tác vụ hàng ngày. Mục đích chính là thay đổi trạng thái vật lý (Physical State) của các thiết bị chấp hành (Actuators). **Ví dụ:** "Bật đèn phòng khách", "Tắt quạt trần đi", "Mở rèm cửa ra", "Giảm điều hòa xuống 24 độ". **Đặc điểm ngôn ngữ:** Cấu trúc câu thường tuân theo quy tắc: Động từ mạnh (Action Verb) + Danh từ chỉ thiết bị (Object) + Bổ ngữ chỉ vị trí/thuộc tính (Location/Attribute). Các động từ thường gặp là: Bật, Tắt, Đóng, Mở, Tăng, Giảm.
- 2. Nhóm lệnh truy vấn thông tin (Query Commands):** Nhóm lệnh này không làm thay đổi trạng thái của thiết bị, mà đóng vai trò yêu cầu hệ thống thu thập và trả về dữ liệu từ các cảm biến môi trường (Sensors) hoặc trạng thái hiện tại của thiết bị. **Ví dụ:** "Nhiệt độ phòng ngủ đang là bao nhiêu?", "Cửa chính đã khóa chưa?", "Độ ẩm hôm nay thế nào?", "Chất lượng không khí có tốt không?". **Đặc điểm ngôn ngữ:** Thường là các câu nghi vấn, chứa các từ để hỏi (Wh-questions) như: bao nhiêu, thế nào, chưa, không, ở đâu.
- 3. Nhóm lệnh giải trí và ngữ cảnh (Entertainment & Context Commands):** Đây là nhóm lệnh phục vụ nhu cầu cá nhân hóa cao cấp, đòi hỏi hệ thống không chỉ hiểu lệnh mà còn phải trích xuất được thông tin chi tiết (Slot Filling/Entity Extraction). **Ví dụ:** "Mở bài hát *Có mình và ta*", "Phát nhạc của *Sơn Tùng MTP*", "Kích hoạt chế độ đi ngủ" (Hệ thống phải tự hiểu ngầm là: Tắt đèn + Khóa cửa + Bật báo động). **Đặc điểm ngôn ngữ:** Chứa các thực thể tên riêng (Named Entities) đa dạng, khó dự đoán trước và đòi hỏi khả năng xử lý linh hoạt của mô hình.

1.4.2. Đặc điểm và Tiêu chuẩn của dữ liệu lệnh tiếng Việt

Để xây dựng được một mô hình học máy chính xác, việc thấu hiểu đặc điểm của dữ liệu đầu vào là yếu tố tiên quyết. Đối tượng nghiên cứu ở đây là "câu lệnh tiếng

Việt", vốn sở hữu những đặc thù phức tạp so với các ngôn ngữ biến hình như tiếng Anh hay tiếng Pháp.

Tiếng Việt là ngôn ngữ đơn lập (Isolating language), từ không biến đổi hình thái theo thì, số hay giống. Ý nghĩa ngữ pháp được thể hiện chủ yếu qua trật tự từ và các hư từ đi kèm.

Khác với bài toán nhận diện hình ảnh (ví dụ: biển số xe) nơi dữ liệu đầu vào là một ma trận điểm ảnh cố định, dữ liệu văn bản là một chuỗi ký tự có độ dài biến thiên (Variable Length). Có lệnh rất ngắn (2 từ): "Bật đèn". Có lệnh rất dài và phức tạp (> 10 từ): "Hãy bật đèn chùm phòng khách màu vàng âm và mở rèm cửa sổ ra giúp tôi nhé". Do đó, hệ thống không thể giới hạn cứng nhắc số lượng ký tự đầu vào. Thay vào đó, chúng ta phải giới hạn về **phạm vi từ vựng (Vocabulary Size)** để đảm bảo mô hình hội tụ tốt trong khuôn khổ tài nguyên phần cứng hạn chế.

- **Về mặt tham số đặc trưng:** Nếu bài toán xử lý ảnh dựa vào các đặc trưng hình học (cạnh, góc, màu sắc), thì bài toán phân loại lệnh dựa vào các tham số thống kê ngôn ngữ:
- **Tần suất từ (Word Frequency):** Mức độ xuất hiện của các từ trong tập dữ liệu.
- **Từ khóa (Keywords):** Những từ mang trọng số thông tin cao quyết định ý định (Ví dụ: "Bật", "Tắt", "Nhạc").
- **Thực thể (Entities):** Các danh từ định danh đối tượng chịu tác động (Ví dụ: "Đèn", "Quạt", tên bài hát).
- **Giới hạn phạm vi nghiên cứu:** Để đảm bảo tính khả thi và hiệu quả cho đề án, nhóm nghiên cứu thiết lập các "thông số điều khiển" để khoanh vùng đối tượng: Tập trung xử lý các câu lệnh đơn và câu ghép đơn giản, tránh các cấu trúc câu phức hợp, đa tầng nghĩa hoặc ẩn dụ.

Ngôn ngữ mục tiêu là Tiếng Việt phổ thông, hạn chế tối đa việc xử lý tiếng lóng (Slang), phương ngữ đặc thù vùng miền hoặc các lỗi chính tả nghiêm trọng. Hệ thống hỗ trợ tập lệnh cho các thiết bị gia đình cơ bản: Hệ thống chiếu sáng (Light), Làm mát (Fan/AC), An ninh (Door/Lock) và Giải trí (Music).

1.5. Giới thiệu mạng neural & Hướng giải quyết

1.5.1. Mạng neural

Mạng nơ-ron nhân tạo (Artificial Neural Network – ANN) là một mô hình tính toán được xây dựng dựa trên cách thức hoạt động của hệ thần kinh sinh học. Mạng bao gồm nhiều đơn vị xử lý gọi là nơ-ron, được liên kết với nhau thông qua các trọng số, cho phép mô hình học và rút trích tri thức từ dữ liệu.

Mỗi nơ-ron thực hiện việc nhận các tín hiệu đầu vào, nhân với các trọng số tương ứng, sau đó cộng thêm một giá trị sai lệch (bias). Kết quả này được đưa qua một hàm kích hoạt để tạo ra đầu ra của nơ-ron. Thông qua quá trình huấn luyện, các trọng số và bias sẽ được điều chỉnh nhằm giảm sai lệch giữa giá trị dự đoán và giá trị thực tế.

Mạng nơ-ron có khả năng mô hình hóa các mối quan hệ phức tạp và phi tuyến trong dữ liệu, nhờ đó được ứng dụng rộng rãi trong nhiều lĩnh vực như phân loại, dự đoán, nhận dạng mẫu, xử lý ảnh và xử lý ngôn ngữ tự nhiên. Trong các mô hình đơn giản nhất, mạng Perceptron một lớp thường được sử dụng cho các bài toán phân loại nhị phân, đóng vai trò nền tảng cho sự phát triển của các kiến trúc mạng nơ-ron sâu hơn.

1.5.2. Hướng giải quyết

Trong kỷ nguyên Big Data, thế giới đã chứng kiến sự bùng nổ của các mô hình ngôn ngữ lớn (Large Language Models - LLMs) như GPT-4, BERT hay LLaMA với khả năng xử lý ngôn ngữ vượt trội. Tuy nhiên, việc áp dụng các mô hình khổng lồ này vào một hệ thống IoT cục bộ (Edge Computing) thường gặp trở ngại về chi phí tính toán, độ trễ và yêu cầu phần cứng.

Với mục tiêu xây dựng một hệ thống "nhẹ", tối ưu hóa tốc độ phản hồi (Low Latency) và đảm bảo quyền riêng tư dữ liệu, nhóm em quyết định không phụ thuộc vào API của bên thứ ba mà tự xây dựng mô hình "In-house". Quy trình giải quyết vấn đề được thiết kế chặt chẽ theo mô hình đường ống (Pipeline) gồm 3 bước chính:

- **Bước 1: Tiền xử lý dữ liệu thô** từ người dùng nhập vào thường chứa nhiều "nhiều". Bước này tương đương với việc lọc và làm sạch nguyên liệu trước khi chế biến. **Chuẩn hóa văn bản:** Chuyển đổi toàn bộ văn bản về dạng chữ thường (lowercase) để máy tính không phân biệt "Bật" và "bật". Loại bỏ các

ký tự đặc biệt, dấu chấm câu dư thừa. **Tách từ (Tokenization):** Đây là bước quan trọng nhất đối với tiếng Việt. Vì tiếng Việt sử dụng dấu cách để ngăn cách các âm tiết chứ không phải từ, nên máy tính cần thuật toán để ghép các từ đơn thành từ ghép có nghĩa (ví dụ: "nhà" + "thông" + "minh" -> "nhà_thông_minh"). Nhóm sử dụng các bộ thư viện chuyên dụng hoặc từ điển tự xây dựng để giải quyết vấn đề này. **Loại bỏ dữ liệu trùng (Deduplication):** Xử lý các mẫu câu giống hệt nhau trong tập huấn luyện để tránh hiện tượng mô hình bị "học vẹt" (Overfitting) vào các câu lệnh phổ biến, đảm bảo tính tổng quát hóa cao.

- **Bước 2: Trích xuất đặc trưng và Vector hóa (Feature Extraction)** Máy tính về bản chất chỉ là những cỗ máy tính toán trên các con số, chúng không hiểu "ngôn ngữ" hay "chữ viết". Bước này đóng vai trò "phiên dịch" từ không gian ngôn ngữ sang không gian vector số học. Nhóm đề xuất sử dụng phương pháp **Bag of Words (Túi từ)**. Đây là kỹ thuật biểu diễn văn bản đơn giản nhưng hiệu quả cho bài toán phân loại câu ngắn. Hệ thống sẽ xây dựng một "Từ điển" (Dictionary) chứa tất cả các từ vựng duy nhất trong tập dữ liệu. Mỗi câu lệnh sau đó sẽ được mã hóa thành một vector có chiều dài bằng kích thước từ điển. Giá trị tại mỗi vị trí trong vector sẽ biểu thị sự xuất hiện (1) hoặc không xuất hiện (0) của từ đó trong câu. Đồng thời, các nhãn ý định (Intent) dạng chữ (như turn_on_light) cũng được mã hóa thành các số nguyên (0, 1, 2...) thông qua kỹ thuật **Label Encoding** để phục vụ cho việc tính toán hàm mất mát.
- **Bước 3: Phân loại ý định bằng Mạng nơ-ron và Thực thi (Classification & Execution)** Đây là giai đoạn quyết định, nơi trí tuệ nhân tạo phát huy tác dụng. **Mô hình hóa (Modeling):** Thay vì dùng các thuật toán máy học cổ điển (như SVM hay Naive Bayes), nhóm sử dụng kiến trúc **Mạng nơ-ron truyền thẳng (Feed Forward Neural Network - FFNN)**. Cấu trúc mạng bao gồm các lớp ẩn (Hidden Layers) với số lượng nơ-ron giảm dần (ví dụ: 128 nodes -> 64 nodes). Sử dụng hàm kích hoạt phi tuyến (như ReLU) giúp mô hình có khả năng học được các mối quan hệ phức tạp giữa các từ trong câu lệnh, điều mà các mô hình tuyến tính không làm được. **Cơ chế phân loại:** Tại lớp đầu ra

(Output Layer), hàm **Softmax** được sử dụng để tính toán xác suất cho từng nhóm ý định. Nhóm có xác suất cao nhất sẽ được chọn làm kết quả dự đoán (Ví dụ: **turn_on_light** với độ tin cậy 98%). **Cơ chế thực thi:** Kết quả dự đoán (Intent) sẽ được chuyển tới ứng dụng di động và gửi đi thông qua giao thức **MQTT**. Backend sẽ đóng vai trò là Publisher, gửi tín hiệu điều khiển tới các Topic tương ứng của thiết bị giả lập

CHƯƠNG 2: PHÂN LOẠI LỆNH BẰNG MẠNG NEURAL NHÂN TẠO (MLP)

2.1. Công nghệ và thư viện sử dụng

Đề tài được phát triển dựa trên mô hình **Client–Server** gồm các thành phần chính:

- **Backend:** Xử lý âm thanh, xử lý dữ liệu, xây dựng **model AI + REST API** và trả kết quả dự đoán intent.
- **Mobile App:** Thu âm, gửi dữ liệu lên backend và điều khiển **module mô phỏng thiết bị nhà thông minh** (giả lập IoT ở backend). Mobile app hoạt động như **remote điều khiển từ xa**, tương tự điều khiển tivi.

Công nghệ Backend: Python 3.x, chính cho AI, xử lý âm thanh và xây dựng API.

Nhóm thư viện	Thư viện	Mục đích sử dụng
Thư viện hỗ trợ API	Flask	Xây dựng REST API xử lý upload âm thanh và trả kết quả dự đoán intent về client.
	Flask-CORS	Cho phép mobile app trên thiết bị khác IP giao tiếp với server.
Thư viện xử lý âm thanh	PyDub	Chuyển đổi định dạng âm thanh (.m4a → .wav); làm sạch và chuẩn hóa âm thanh đầu vào.
	speech_recognition	Nhận diện giọng nói tiếng Việt; tối ưu cho các lệnh ngắn như “bật quạt”, “mở đèn”, “tắt cửa”...
	edge_tts	Chuyển phản hồi từ văn bản thành giọng nói tiếng Việt (giọng Bắc).
Thư viện xử lý tính toán train mô hình	PyTorch	Xây dựng, huấn luyện và load mô hình Neural Network.
	Scikit-learn	Vectorizer (CountVectorizer/TF-IDF) chuyển văn bản → vector số;

Nhóm thư viện	Thư viện	Mục đích sử dụng
		LabelEncoder mã hóa nhãn intent; xử lý dữ liệu khi dự đoán.
	Pandas & NumPy	Đọc dataset (dataset1.csv) và xử lý dữ liệu dạng bảng.
Một số thư viện hỗ trợ khác	joblib	Lưu mô hình, train 1 lần, sử dụng nhiều lần
	tqdm	Tạo thanh loading khi train trực quan
	Paho	Xử lý giả lập phần cứng
	Os	Xử lý các phần liên quan đến hệ thống

Mobile App (React Native – Expo): React Native 0.81.5, React 19.1.0, Expo SDK 54, Expo giúp lập trình mobile nhanh, dễ build, không cần Android Studio trong giai đoạn phát triển.

Thư viện	Mục đích sử dụng
expo-av (~16.0.7)	Thu âm giọng nói; tạo, bắt đầu, dừng, lưu file âm thanh (.m4a); gửi file lên API backend.
expo-file-system (~19.0.17)	Lưu file thu âm tạm thời; lấy đường dẫn file để gửi đi.
expo-router	Routing dạng thư mục; tối ưu cấu trúc app.
expo-image, expo-haptics, expo-splash-screen, expo-status-bar	Hiển thị hình ảnh, rung nút, màn hình chờ, tùy chỉnh thanh trạng thái.
react-native-web, react-dom	Hỗ trợ chạy thử trên PC.
TypeScript	Tăng độ an toàn mã nguồn; tự động gợi ý và kiểm tra lỗi.

Lý do chọn các công nghệ:

- **Flask + Flask-CORS:** Triển khai API AI thời gian thực đơn giản, nhanh; hỗ trợ mobile giao tiếp từ thiết bị khác IP.

- **PyTorch:** Mạnh mẽ và linh hoạt trong xây dựng và chạy mô hình Neural Network.
- **React Native + Expo:** Phát triển ứng dụng mobile đa nền tảng nhanh chóng, dễ kiểm thử.
- **TypeScript:** Giảm lỗi khi dự án có nhiều module; bảo vệ mã nguồn.
- **PyDub + STT/TTS:** Xử lý giọng nói tiếng Việt hiệu quả, phản hồi nhanh các lệnh ngắn.
- **Vectorizer + MLP:** Nhẹ, đủ cho dự đoán intent real-time.
- **MQTT (control_devices):** Mô phỏng Smart Home IoT chân thực.
- **Thư viện UI frontend (expo-image, expo-haptics...):** Cải thiện trải nghiệm người dùng.
- **Module mở rộng backend (music_player, weather_api):** Hỗ trợ các chức năng phụ trợ, giúp dự án hoàn chỉnh hơn.

2.2. Kỹ thuật sử dụng

MLP (Multi-Layer Perceptron) là một dạng mạng nơ-ron nhân tạo truyền thẳng (feed-forward neural network), bao gồm nhiều lớp nơ-ron xếp chồng lên nhau. Một mô hình MLP cơ bản gồm ba loại lớp: lớp đầu vào (input layer), một hoặc nhiều lớp ẩn (hidden layer) và lớp đầu ra (output layer). Các nơ-ron giữa các lớp được kết nối đầy đủ thông qua các trọng số.

Trong MLP, mỗi nơ-ron thực hiện phép tính tuyến tính trên các giá trị đầu vào, cộng thêm bias và đưa kết quả qua hàm kích hoạt phi tuyến như sigmoid, ReLU hoặc tanh. Việc sử dụng các hàm kích hoạt phi tuyến giúp MLP có khả năng học và biểu diễn các mối quan hệ phức tạp trong dữ liệu mà các mô hình tuyến tính không thể thực hiện được.

Quá trình huấn luyện MLP thường được thực hiện bằng thuật toán lan truyền ngược (backpropagation), trong đó sai số ở đầu ra được lan truyền ngược trở lại các lớp trước để cập nhật trọng số nhằm tối thiểu hóa hàm mất mát. Nhờ khả năng học mạnh mẽ và tính linh hoạt, MLP được ứng dụng rộng rãi trong các bài toán phân loại, hồi quy và nhận dạng mẫu.

Trong đề tài này, nhóm sử dụng kỹ thuật Multilayer Perceptron (MLP) – một dạng mạng Neural nhân tạo truyền thẳng (Feedforward Neural Network) – để giải quyết bài toán phân loại lệnh điều khiển thiết bị thông minh. MLP là kỹ thuật học sâu cơ bản nhưng hiệu quả, phù hợp cho dữ liệu nhỏ và đầu vào dạng vector đặc trưng thu được từ BoW.

2.2.1. Tổng quan về MLP

Multilayer Perceptron (MLP) bao gồm ba thành phần chính: lớp đầu vào, các lớp ẩn và lớp đầu ra. Mỗi thành phần đảm nhiệm một vai trò riêng trong quá trình xử lý và học dữ liệu của mô hình.

- **Lớp đầu vào (Input Layer)** là lớp tiếp nhận dữ liệu ban đầu từ bên ngoài. Mỗi nơ-ron trong lớp này tương ứng với một đặc trưng của mẫu dữ liệu đầu vào. Lớp đầu vào không thực hiện tính toán mà chỉ có nhiệm vụ truyền các giá trị đặc trưng sang các lớp tiếp theo.
- **Các lớp ẩn (Hidden Layers)** nằm giữa lớp đầu vào và lớp đầu ra, có thể gồm một hoặc nhiều lớp. Mỗi nơ-ron trong lớp ẩn thực hiện phép biến đổi tuyến tính kết hợp với hàm kích hoạt phi tuyến. Các lớp ẩn giúp mô hình học được các đặc trưng và mối quan hệ phức tạp trong dữ liệu mà các mô hình đơn giản không thể biểu diễn.
- **Lớp đầu ra (Output Layer)** là lớp cuối cùng của mạng, chịu trách nhiệm tạo ra kết quả dự đoán. Số lượng nơ-ron trong lớp đầu ra phụ thuộc vào bài toán cụ thể, chẳng hạn như một nơ-ron cho bài toán phân loại nhị phân hoặc nhiều nơ-ron cho bài toán phân loại đa lớp. Hàm kích hoạt của lớp này thường được lựa chọn phù hợp với dạng đầu ra mong muốn.

Dữ liệu di chuyển theo một chiều duy nhất từ đầu vào đến đầu ra, không có vòng lặp, do đó MLP thuộc nhóm mạng Feedforward Neural Network.

Mỗi neuron thực hiện phép tính: $[y = f(Wx + b)]$ Trong đó:

+ (x) : đầu vào

+ (W) : trọng số

+ (b) : *bias*

+ (f) : *hàm kích hoạt*

2.2.2. Lý do lựa chọn kỹ thuật MLP

Kỹ thuật MLP được lựa chọn dựa trên các ưu điểm:

- Phù hợp với phân loại câu lệnh ngắn, đặc biệt là lệnh giọng nói dạng: “bật quạt”, “mở đèn”, “phát nhạc”...
- Hoạt động tốt với vector hóa Bag of Words (BoW).
- Không đòi hỏi tài nguyên cao, xử lý được trên máy tính cá nhân.
- Độ chính xác tốt cho các bài toán có số lượng intent nhỏ.
- So với các mô hình phức tạp hơn (LSTM, BERT), MLP nhẹ hơn và phù hợp với hệ thống điều khiển thiết bị gia đình.

2.2.3. Cấu trúc mô hình MLP trong đề tài

1. Lớp đầu vào:

Nhận vector đặc trưng sinh từ kỹ thuật Bag of Words (BoW).

Kích thước đầu vào phụ thuộc số lượng từ trong bộ từ điển (vocabulary).

2. Các lớp ẩn:

Số lớp: 1–2 lớp

Số neuron: 64–128

Hàm kích hoạt: ReLU

Vai trò: học các biểu diễn phi tuyến, tách biệt các loại lệnh.

3. Lớp đầu ra:

Số neuron bằng số intent (mở quạt, tắt quạt, mở đèn...).

Hàm kích hoạt: Softmax để đưa ra xác suất của từng lớp.

2.2.4. Hàm mất mát và phương pháp tối ưu

- Hàm mất mát: CrossEntropyLoss (CrossEntropyLoss phù hợp với phân loại nhiều lớp, còn Adam giúp mô hình hội tụ nhanh).
- Optimizer: Adam
- Learning rate: 0.001 – 0.0005

2.2.5. Cơ chế hoạt động của kỹ thuật MLP

Quá trình phân loại diễn ra theo các bước:

1. Nhận đầu vào: câu lệnh được biểu diễn bằng vector BoW.
2. Lan truyền tiến (Forward Propagation): dữ liệu đi qua các lớp ẩn.
3. Kết luận: lớp Softmax chọn intent có xác suất cao nhất.

Trong quá trình huấn luyện, mô hình sử dụng lan truyền ngược (Backpropagation) để cập nhật trọng số dựa trên sai số.

2.2.6. Ưu điểm và hạn chế

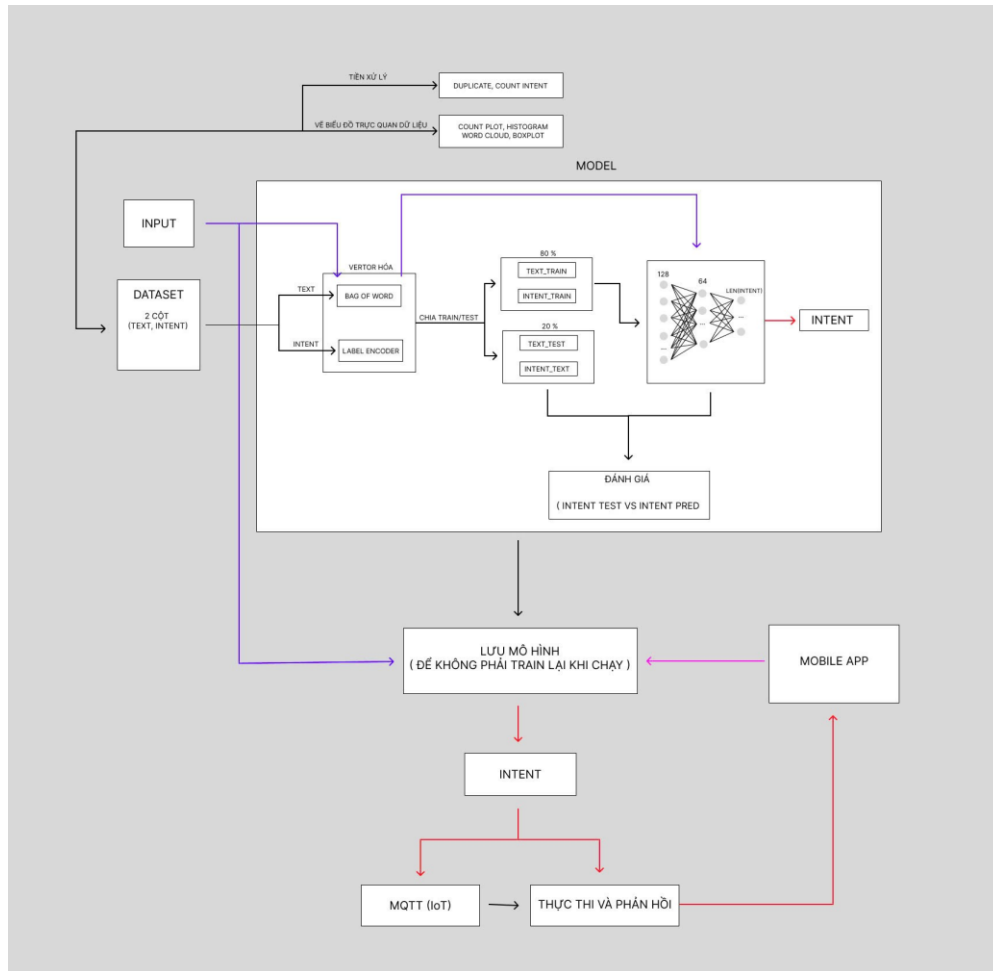
Ưu điểm:

- Mô hình nhẹ, dễ huấn luyện.
- Hoạt động tốt với dữ liệu nhỏ.
- Dễ triển khai trong thực tế.
- Dễ mở rộng thêm các lệnh mới.

Hạn chế:

- Không xử lý ngữ cảnh dài.
- Khả năng hiểu ngôn ngữ tự nhiên hạn chế hơn các mô hình hiện đại.
- Phụ thuộc nhiều vào chất lượng biểu diễn Bag of Words.

2.3. Quy trình thực hiện

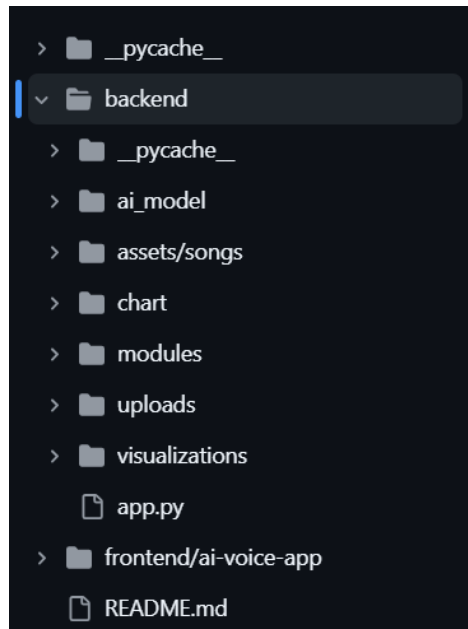


Hình 1: Sơ đồ luồng hoạt động của dự án

Trong chương này mô tả chi tiết quy trình thực hiện bài toán phân loại lệnh và điều khiển cho hệ thống nhà thông minh. Các bước thực hiện bao gồm

1. Chuẩn bị dữ liệu
2. Tiền xử lý
3. Huấn luyện mô hình
4. Test và đánh giá độ chính xác của mô hình
5. Tích hợp vào mobile app + giả lập phần cứng.

Toàn bộ code và tài nguyên tham chiếu trong chương này được triển khai trong repository của nhóm. Cấu trúc thư mục như sau:



Hình 2: Cấu trúc thư mục dự án

Giải thích:

- ai_model: Thư mục chứa các files code dùng để train mô hình, xử lý giả lập phần cứng, v.v.
- assets/songs: Thư mục chứa các files bài hát ở định dạng mp4.
- chart: Thư mục chứa các biểu đồ dùng để tiền xử lý và các phần xử lý chức năng.
- modules: Thư mục chứa files tiền xử lý, vẽ biểu đồ, v.v.
- uploads: Thư mục lưu âm thanh dùng để test giọng nói định dạng files m4a, wav.
- visualizations: Thư mục chứa code để trực quan hóa dữ liệu.
- app.py: Là file chạy chương trình chính.
- Frontend: Thư mục chứa các files code giao diện mobile app.
- README.md: File hướng dẫn setup môi trường và sử dụng.

2.3.1. Chuẩn bị dữ liệu (Dataset)

Mục tiêu: Thu thập một tập câu lệnh (text) đại diện cho các hành vi người dùng có thể ra lệnh cho hệ thống nhà thông minh, đảm bảo đa dạng cách nói tự nhiên và xây dựng bảng nhãn (intent) phù hợp cho bài toán phân loại đa lớp.

Nguồn dữ liệu và cấu trúc dataset:

- Nhóm thu thập dữ liệu thủ công từ các câu nói hàng ngày đồng thời kết hợp làm sạch dữ liệu để mô hình học tốt nhất có thể. file dataset.csv chứa hàng nghìn câu lệnh mẫu (ví dụ: “bật nhạc”, “tắt nhạc”, “mở cửa”, “đóng cửa”, “mở quạt”, “tắt quạt”, “bật đèn”, “tắt đèn”, ...). Đây là nguồn chính cho việc huấn luyện mô hình NLP .
- Tập dataset được lưu vào file dạng bảng, 2 cột (text, intent) phù hợp để test demo nhanh đồng thời dễ dàng mở rộng dataset khi cần nâng cấp hệ thống để đưa vào hoạt động thực tế.

Với đề tài này, nhóm lựa chọn phương pháp thu thập dữ liệu thủ công để test demo nhanh và dễ kiểm soát các câu lệnh khi train/test mô hình.

Lớp (intent) mục tiêu:

Từ tập câu lệnh mẫu, nhóm xây tập intent chính gồm các lớp điển hình cho nhà thông minh, Ví dụ:

- play_music, stop_music
- open_door, close_door
- open_fan, close_fan
- turn_on_lights, turn_off_lights
- weather, great (thăm dò trạng thái)
- ...

(chi tiết lớp và mapping label được lưu/thu lại trong label_encoder.pkl).

Việc lựa chọn nhãn như vậy là phù hợp với thực tế và nhu cầu đối với dự án vì đề tài này tập trung vào việc phân loại lệnh và thực thi lệnh ở ngôi nhà thông minh.

Phân lớp như vậy còn dễ dàng mở rộng và phát triển tiếp.

Dự trữ và chia bộ dữ liệu:

Sau khi làm sạch và gán nhãn, dữ liệu được chia thành: training, testing. Việc chia này được thực hiện sao cho phân bố các lớp (class distribution) tương đối cân bằng; nếu không cân, cần áp dụng kỹ thuật upsampling/undersampling (xem phần Preprocessing).

Việc chia dữ liệu là cần thiết để mô hình đánh giá được mô hình học có tốt hay là không. Tập train sẽ chứa các dữ liệu để huấn luyện mô hình còn tập test sẽ chứa dữ liệu mà mô hình chưa thấy bao giờ (chưa được học bao giờ) để đem ra dự đoán và đánh giá.

1	text	intent
2	bật nhạc	play_music
3	bật nhạc ngay cho tôi	play_music
4	bật nhạc giúp tôi nào	play_music
5	bật nhạc đi nào	play_music
6	bật nhạc đi	play_music

Hình 3: Dữ liệu dạng bảng 2 cột (text - intent)

Tổng kết:

Việc xây dựng dataset theo cách này giúp dữ liệu bám sát các tình huống sử dụng thực tế, đúng ngữ cảnh và giọng nói địa phương của người Việt. Ngoài ra, việc sử dụng STT thay vì xử lý đặc trưng âm thanh (MFCC) giúp giảm độ phức tạp của mô hình và phù hợp với phạm vi môn học.

Tổng số mẫu sau khi hoàn thiện đạt mức đủ để mô hình học được cấu trúc ngôn ngữ cơ bản, đồng thời vẫn đảm bảo thời gian huấn luyện ngắn và phù hợp với môi trường chạy thử.

2.3.2. Xử lý dữ liệu (Preprocessing)

Mục tiêu:

Vì dữ liệu ban đầu là chữ tiếng việt nên trước khi đưa vào mô hình cần dùng các kỹ thuật tiền xử lý văn bản trong NLP (xử lý ngôn ngữ tự nhiên). Phần này nhóm sử dụng kỹ thuật BoW (Bag of Word) vì nó đơn giản, nhanh và đặc biệt là phù hợp với bài toán phân loại lệnh này, một bài toán không quá đòi hỏi về khả năng hiểu ngữ nghĩa sâu của câu lệnh.

Ngoài BoW, vì muốn trực quan và sát với thực tế nhất có thể nên nhóm có sử dụng thêm phần ghi âm bằng giọng nói để nhận lệnh trực tiếp bằng giọng nói rồi sau đó dùng kỹ thuật STT tức là chuyển âm thanh thành văn bản.

Preprocessing gồm hai luồng chính vì dự án có cả audio-frontend và text-backend:

- Luồng audio → text: người dùng nói → ghi âm → STT → văn bản → API
- API → Luồng text → vector (BoW) → input cho MLP

Ngoài ra để mô hình học tốt hơn nhóm còn sử dụng thêm các kỹ thuật như là stopwords, subwords, ...

Pipeline chi tiết (theo triển khai trong repo)

Bước 1: Ghi âm và chuyển sang text (audio → text)

File: input_sound_processor.py chứa class Recorder, STT, TTS.

- Recorder thu âm 5s ở 44.1 kHz, chuyển sang int16 và có thể lưu file WAV hoặc file MP3.
- STT dùng speech_recognition (Google API) để convert audio → text; có tinh chỉnh tham số: energy_threshold, pause_threshold, phrase_threshold để nhận tiếng Việt tốt hơn.
- TTS dùng edge_tts để phát phản hồi (cấu hình voice: vi-VN-HoaiMyNeural, vi-VN-NamMinhNeural).

Giá trị output của luồng này là câu văn bản (Tiếng Việt), tương ứng một mẫu text để đưa vào pipeline NLP.

Bước 2: Text cleaning (làm sạch văn bản)

Mục tiêu: Giảm nhiễu ngôn ngữ tự nhiên, chuẩn hóa để mô hình tập trung vào từ khóa intent.

Các thao tác gồm:

- chuyển chữ hoa → chữ thường,
- loại bỏ dấu câu, ký tự đặc biệt, số thừa,
- gỡ bỏ stop-words không cần thiết (tùy chọn),

- loại bỏ/giảm các từ cảm thán hoặc từ phụ (ví dụ “giùm”, “nhé”, “nào”,...) nếu gây nhiễu cho phân lớp,
- chuẩn hóa Unicode (nếu cần) và chuẩn hóa các biến thể viết tắt.

Bước 3: Tokenization & normalization

- Tokenization bằng tokenizer cơ bản (split whitespace) hoặc tokenizer nâng cao (khi dùng PhoBERT thì dùng AutoTokenizer tương ứng).
- Có thể ứng dụng stemming/lemmatization (với tiếng Việt, thường ít dùng; thay bằng xử lý từ ghép và chuẩn hóa).

Bước 4: Vector hóa (text → numeric)

- File: vectorizer.pkl — đại diện cho một trình vector hóa đã lưu (có thể là CountVectorizer hoặc TfidfVectorizer từ scikit-learn).
- Cách làm: sau làm sạch, áp dụng vectorizer.transform(texts) để thu được ma trận dạng ($n_samples \times n_features$).
- Ưu điểm: nhẹ, chạy nhanh trên CPU; phù hợp cho MLP nhỏ.
- Lưu ý: cần lưu vectorizer.pkl cùng label_encoder.pkl để dịch ngược label khi predict (repo đã lưu hai file này).

Bước 5: Mã hóa nhãn (Label encoding)

- File label_encoder.pkl chứa mapping giữa intent string ↔ integer (ví dụ "play_music" → 1).
- Trong huấn luyện: labels → one-hot (nếu dùng nn.CrossEntropyLoss thì chỉ cần integer labels).

Bước 7: Chia bộ dữ liệu

- Dữ liệu được shuffle và chia train/test (80/20).
- Khi dùng cross-validation (nếu cần) hoặc K-fold để tune hyperparameters, đảm bảo quy trình vectorizer / scaler được fit chỉ trên tập train để tránh data leakage.

Ghi chú thực thi trong repo

- `check_dataset.py` dùng để kiểm tra tính đồng nhất và thống kê dataset (số mẫu mỗi label).
- `build_model.py` và `build_model_v2.py` chứa logic huấn luyện (v1: vectorizer + MLP; v2: PhoBERT + MLP/Pytorch). Trong README, nhóm khuyến nghị tải PhoBERT offline (lưu trong `phobert-base-offline/`) nếu muốn chạy phiên bản transformer.

Hình 4: Biểu đồ trực quan hóa dữ liệu

Mục tiêu: Thiết kế và huấn luyện một mô hình phân loại intent có hiệu năng tốt trên dữ liệu câu lệnh tiếng Việt, chạy được trong điều kiện frontend mobile + backend nhẹ. Thiết kế mô hình — hai luồng chính trong repo

- Input: vector BoW/Count (vectorizer.pkl).
- Mạng: MLP nhỏ (fully-connected feed-forward neural network). Một kiến trúc tiêu biểu:
 - Input layer: chiều = số feature (ví dụ 1.000–10.000 tùy vocab size)
 - Hidden layer 1: 128 neuron, activation = ReLU
 - Hidden layer 2: 64 neuron, activation = ReLU (tùy phiên bản)
 - Output layer: softmax với C neuron (C = số lớp intent)
- Loss: Cross-Entropy (nn.CrossEntropyLoss hoặc sklearn tương đương).

- Optimizer: Adam (learning rate $\approx 1e-3$) hoặc SGD (khả năng tinh chỉnh).
- Regularization: Dropout (ví dụ $p=0.3$) nếu model overfit; L2 weight decay nếu cần.
- Lưu model: intent_classifier.pkl (phiên bản sklearn) hoặc intent_model.pt (phiên bản PyTorch).

b) Huấn luyện (training workflow)

- Chuẩn bị: load vectorizer.pkl (v1) hoặc phobert tokenizer + model (v2); load label_encoder.pkl.
- Batching: batch size (ví dụ 16–64), epochs (ví dụ 10–30) tùy tài nguyên; trong repo có cấu hình mặc định trong build_model.py / build_model_v2.py.
- Theo dõi: lưu loss/accuracy theo epoch, vẽ đồ thị (chart/chart.png được repo sinh ra).
- Checkpoint: lưu model tốt nhất theo metric validation (early stopping nếu cần).
- Kết quả lưu: intent_classifier.pkl (sklearn) hoặc intent_model.pt cho inference.

Lý thuyết ngắn gọn (MLP)

- MLP là mạng neural feed-forward nhiều lớp, mỗi lớp thực hiện:

$$\mathbf{h}^{(l)} = \sigma(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)})$$

- với σ là hàm kích hoạt (ReLU/ Tanh/ Sigmoid). Tầng cuối dùng softmax để sinh phân phối xác suất trên các lớp:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

- Hàm mất mát dùng cross-entropy giữa phân phối dự đoán và one-hot label.

a. Ghi chú thực tế

- Khi dùng TF-IDF + MLP, lưu ý kích thước feature vector lớn → khuyến nghị giảm dim bằng max_features hoặc TruncatedSVD nếu cần.
- Khi dùng PhoBERT, cần GPU để huấn luyện nhanh; repo đã lưu intent_model_gpu.pt (tên cho thấy training/ inference GPU).

```
# Mạng nơ-ron đơn giản (MLP)
class TorchClassifier(nn.Module):
    def __init__(self, input_size, num_classes):
        super(TorchClassifier, self).__init__()
        self.net = nn.Sequential(
            nn.Linear(input_size, 128),
            nn.ReLU(),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, num_classes)
        )

    def forward(self, x):
        return self.net(x)
```

Hình 5: Minh họa mạng neural MLP

2.3.4. Dự đoán và đánh giá

Đánh giá hiệu năng mô hình phân lớp intent bằng các metric tiêu chuẩn để kiểm tra độ chính xác, khả năng tổng quát hoá và lỗi hay gặp.

Metric sử dụng:

- Accuracy: tỉ lệ dự đoán đúng trên tổng số mẫu test.
- Precision, Recall, F1-Score (từng lớp và macro/micro) — đặc biệt quan trọng với dữ liệu có lớp không cân bằng.
- Confusion matrix: để quan sát nhầm lẫn giữa các cặp intent (ví dụ play_music ↔ open_door có thể ít nhầm; turn_on_lights ↔ play_music hiếm thấy).
- ROC / PR Curve: nếu cần xét từng nhãn theo ngưỡng (ít dùng cho multi-class có softmax).

```
# ----- ĐÁNH GIÁ -----
def evaluate(self):
    self.model.eval()
    y_true, y_pred = [], []
    with torch.no_grad():
        for X_batch, y_batch in self.test_loader:
            X_batch = X_batch.to(self.device)
            outputs = self.model(X_batch)
            preds = torch.argmax(outputs, dim=1)
            y_true.extend(y_batch.numpy())
            y_pred.extend(preds.cpu().numpy())
```

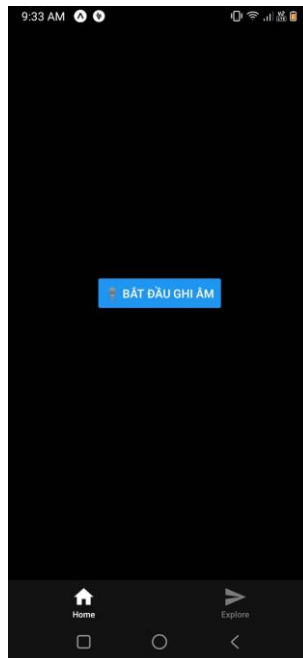
Hình 6: Code đánh giá mô hình

2.3.5. Xây dựng Mobile App và giả lập phần cứng

Mục tiêu: Xây dựng mobile app làm giao diện người dùng cho phép: thu âm, gửi lệnh, nhận kết quả, phát nhạc hoặc điều khiển thiết bị giả lập; đồng thời mô phỏng phần cứng (thiết bị) để kiểm thử end-to-end.

Kiến trúc tích hợp (theo Expo Go):

- Frontend (Mobile App): nằm trong frontend/ai-voice-app/ (Expo Go chứa mã nguồn frontend). App thực hiện:
 - Ghi âm người dùng (hoặc upload file audio),
 - Gọi STT local hoặc API (module input_sound_processor.py dùng speech_recognition để convert),
 - Gửi text hoặc feature lên backend qua HTTP API hoặc WebSocket.
- Backend: API endpoint nhận text hoặc audio:
 - Nếu nhận audio: backend gọi STT → text → vectorize → model predict (sử dụng predict.py / predict_v2.py).
 - Nếu nhận text: backend trực tiếp chạy vectorizer/embedding → predict.
- Model serving: file predict.py và predict_v2.py chứa logic load vectorizer.pkl, label_encoder.pkl, intent_classifier.pkl/intent_model.pt và thực hiện inference; phần này có thể triển khai dưới dạng Flask/FastAPI để app gọi.



Hình 7: Giao diện mobile app

Với giao diện mobile app nhóm tối ưu vào khả năng dễ sử dụng, phản hồi nhanh và cần đơn giản để tiếp cận với đa số người dùng.

Quy trình giả lập phần cứng:

- Mô phỏng thiết bị: backend chứa module `music_player.py` (trong `version_2`) và lệnh điều khiển file trên server (ví dụ phát file MP3 trong `songs/`) để giả lập hành vi thiết bị (bật/tắt nhạc, bật/tắt quạt/đèn).
- Giao tiếp App ↔ Thiết bị giả lập: app gửi intent → backend mapping intent → hành động (kích hoạt giả lập thiết bị).
- Feedback: TTS (`edge_tts`) được dùng để phát câu trả lời từ hệ thống khi app cần phản hồi người dùng.

Trải nghiệm UX: Người dùng nhấn nút “Nói” trên app → app ghi âm 3–5 giây (Recorder).

App gửi audio → backend (nếu STT ở server) hoặc local STT (nếu tích hợp trên thiết bị).

Backend chuyển audio → text (STT), tiền xử lý text, chạy model → trả về label.

App hiển thị hành động dự đoán và/hoặc backend thực thi hành động (phát nhạc, mở quạt).

Hệ thống phát phản hồi bằng TTS nếu cần.

2.3.6. Tối ưu mô hình

Để tối ưu mô hình hiệu suất cao nhất nhóm sử dụng các phương pháp sau:

- Clean code: code sạch, không rườm rà để code dễ đọc, dễ triển khai
- Lập trình bất đồng bộ: Xử lý nhiều tác vụ cùng một lúc trong cùng một khoảng thời gian, tận dụng tối đa xử lý đa nhân, đa luồng của CPU, GPU.
- Batch: Chia tập dữ liệu thành các batch để tối ưu phần training giúp mô hình học nhanh, hiệu quả hơn. Tránh học tử.
- ...

2.4. Kết luận

Nhóm 2 chúng em đã thu thập và tổ chức tập câu lệnh (file dataset1.csv, dataset.csv), chuẩn hóa và vector hóa (sử dụng vectorizer.pkl hoặc PhoBERT embeddings).

- Hai hướng mô hình: lightweight TF-IDF + MLP (dễ chạy realtime trên CPU) và PhoBERT + classifier (chính xác hơn nhưng nặng hơn). Cả hai đã được triển khai/ lưu trong repo (intent_classifier.pkl, intent_model.pt, intent_model_gpu.pt).
- Mobile app và module giả lập phần cứng tích hợp chặt chẽ để test end-to-end; input_sound_processor.py, predict.py/predict_v2.py cung cấp workflow inference.
- Đánh giá dùng accuracy, precision/recall/F1 và confusion matrix; đồ thị huấn luyện được lưu trong chart/chart.png.

CHƯƠNG 3: KẾT QUẢ

Đề tài “**Phát triển hệ thống phân loại lệnh ứng dụng vào ngôi nhà thông minh**” tập trung vào việc nghiên cứu và triển khai một hệ thống có khả năng nhận dạng, phân loại và phản hồi các lệnh điều khiển từ người dùng. Hệ thống được thiết kế sử dụng mạng nơ-ron truyền thẳng đa lớp (MLP – Multilayer Perceptron) – một trong những mô hình học máy cơ bản nhưng mạnh mẽ, có khả năng học các đặc trưng phức tạp từ dữ liệu đầu vào và đưa ra quyết định phân loại chính xác. Việc áp dụng MLP giúp hệ thống có thể xử lý các lệnh dạng văn bản, âm thanh hoặc tín hiệu từ các cảm biến một cách linh hoạt, đồng thời dễ dàng mở rộng khi bổ sung các loại lệnh mới.

Hệ thống được phát triển với các mục tiêu cụ thể sau:

1. **Tự động hóa điều khiển thiết bị:** Hệ thống nhằm giảm thiểu các thao tác thủ công của người dùng, giúp các thiết bị trong ngôi nhà có thể tự động hoạt động theo lệnh đã được phân loại. Việc tự động hóa này không chỉ mang lại sự tiện nghi mà còn tối ưu hóa quá trình quản lý năng lượng, giảm thiểu lãng phí và nâng cao hiệu quả vận hành của toàn bộ hệ thống.
2. **Đảm bảo tính ổn định và độ chính xác:** Một yêu cầu quan trọng của hệ thống là khả năng phân loại lệnh đúng và ổn định, ngay cả khi dữ liệu đầu vào có sự biến đổi nhẹ hoặc nhiễu. Mục tiêu này được thực hiện thông qua việc thiết kế mô hình MLP với các lớp ẩn và hàm kích hoạt phù hợp, kết hợp với quá trình huấn luyện và kiểm tra chặt chẽ trên dữ liệu mô phỏng. Độ chính xác cao đảm bảo các thiết bị phản hồi đúng hành động, từ đó nâng cao trải nghiệm người dùng và tin cậy của hệ thống.
3. **Tạo nền tảng cho nghiên cứu và ứng dụng mở rộng:** Hệ thống được xây dựng với khả năng mở rộng cao. Kiến trúc mô hình MLP cho phép bổ sung thêm nhiều loại lệnh, tích hợp với các thiết bị IoT mới hoặc mở rộng các phương thức điều khiển khác như giọng nói, cử chỉ hoặc dữ liệu cảm biến đa chiều. Điều này không chỉ phục vụ cho nghiên cứu nâng cao về trí tuệ nhân tạo trong nhà thông minh mà còn cung cấp cơ sở để triển khai các

giải pháp thực tế, hướng tới một môi trường sống thông minh, tiện nghi và thân thiện với người dùng.

Ngoài ra, đề tài còn đặt mục tiêu xây dựng quy trình tổng thể cho hệ thống, bao gồm: thu thập dữ liệu lệnh, tiền xử lý dữ liệu, thiết kế mô hình MLP, huấn luyện và đánh giá mô hình, cũng như tích hợp thử nghiệm trong môi trường mô phỏng. Quy trình này giúp đảm bảo tính khoa học và khả năng lặp lại trong nghiên cứu, đồng thời làm cơ sở cho các bước phát triển tiếp theo như tối ưu mô hình, mở rộng dữ liệu và triển khai thực tế trên thiết bị IoT.

3.1. Kết quả đạt được

Trong quá trình triển khai đề tài “*Phát triển hệ thống phân loại lệnh ứng dụng vào ngôi nhà thông minh*”, nhóm đã đạt được những kết quả quan trọng về cả mặt kỹ thuật, ứng dụng thực tiễn và giá trị học thuật. Các kết quả này chứng minh tính khả thi của việc sử dụng mạng nơ-ron truyền thẳng đa lớp (MLP) trong việc phân loại lệnh điều khiển cho các thiết bị nhà thông minh.

3.1.1. Video Demo & source code

Link: <https://youtu.be/o7hIQ3c87zw>

Link GitHub: https://github.com/thang170725/tri_tue_nhan_tao.git

3.1.1. Kết quả về mặt kỹ thuật

- **Thiết kế mô hình MLP phù hợp:** Hệ thống được xây dựng dựa trên mô hình MLP với số lớp ẩn, số neuron và hàm kích hoạt được điều chỉnh hợp lý. Việc lựa chọn kiến trúc này đảm bảo rằng mô hình có khả năng học và nhận dạng các đặc trưng của lệnh một cách chính xác, đồng thời có thể phân loại các lệnh cơ bản một cách ổn định. Các tham số như tốc độ học, số epoch và hàm mất mát được tối ưu hóa thông qua thử nghiệm thực tế, nhằm đạt hiệu suất cao nhất mà vẫn tránh hiện tượng quá khớp (overfitting).
- **Tiến trình huấn luyện chuẩn hóa:** Quy trình huấn luyện mô hình tuân thủ các bước cơ bản và khoa học của học máy:
 - **Tiền xử lý dữ liệu:** Loại bỏ nhiễu, chuẩn hóa định dạng dữ liệu, chuẩn hóa các lệnh đầu vào về dạng phù hợp với MLP.

- **Chuẩn hóa dữ liệu:** Dữ liệu được chuẩn hóa về phạm vi nhất định để mô hình học nhanh hơn và ổn định hơn.
- **Chia tập dữ liệu:** Tách dữ liệu thành tập huấn luyện, tập kiểm tra và tập đánh giá để đảm bảo mô hình không bị học quá mức trên tập dữ liệu huấn luyện.
- **Huấn luyện mô hình:** MLP được huấn luyện với thuật toán lan truyền ngược (backpropagation) để cập nhật trọng số nhằm giảm lỗi phân loại.
- **Đánh giá hiệu năng:** Sử dụng các chỉ số như độ chính xác (accuracy), ma trận nhầm lẫn (confusion matrix) và tỷ lệ lỗi để đánh giá hiệu quả mô hình.
- **Hiệu năng và khả năng phân loại:** Kết quả thử nghiệm cho thấy mô hình MLP có khả năng phân loại chính xác các lệnh cơ bản, đạt mức độ ổn định cao, không bị dao động nhiều khi thử nghiệm với dữ liệu khác nhau. Điều này chứng tỏ kiến trúc MLP phù hợp với bài toán phân loại lệnh trong môi trường mô phỏng nhà thông minh.
- **Khả năng mở rộng và điều chỉnh:** Hệ thống được thiết kế linh hoạt, dễ dàng mở rộng bằng cách thêm các lệnh mới hoặc điều chỉnh tham số mô hình. Việc này giúp hệ thống có khả năng thích ứng với các thay đổi về yêu cầu và các loại lệnh mới trong tương lai mà không cần thiết kế lại toàn bộ mô hình.

3.1.2. Kết quả về mặt ứng dụng

- **Phân loại lệnh thành các nhóm điều khiển:** Hệ thống MLP có khả năng phân loại các lệnh thành các nhóm điều khiển thiết bị khác nhau như: điện, quạt, đèn, điều hòa, hệ thống an ninh, v.v. Sau khi phân loại, lệnh được chuyển thành tín hiệu điều khiển chính xác cho từng thiết bị tương ứng, đảm bảo các thiết bị thực hiện đúng hành động mong muốn của người dùng.
- **Khả năng ứng dụng thực tế:** Mô hình MLP chứng minh tính khả thi của việc áp dụng trí tuệ nhân tạo trong hệ thống nhà thông minh. Việc phân loại tự động các lệnh giúp giảm thao tác thủ công, nâng cao trải nghiệm người dùng và tối ưu hóa vận hành hệ thống. Điều này mở ra khả năng tích hợp vào các hệ thống IoT thực tế, từ các thiết bị đơn lẻ đến toàn bộ ngôi nhà thông minh.

- **Cơ sở cho các giải pháp mở rộng:** Quy trình xây dựng hệ thống cung cấp nền tảng để triển khai các mô hình AI khác hoặc kết hợp thêm nhiều phương thức điều khiển, chẳng hạn như: nhận dạng giọng nói, điều khiển bằng cử chỉ, dữ liệu cảm biến môi trường, giúp hệ thống trở nên linh hoạt, thông minh và thân thiện với người dùng hơn.

3.2.3. Kết quả về học thuật

- **Áp dụng kiến thức lý thuyết vào thực tiễn:** Nhóm đã vận dụng các kiến thức về mạng nơ-ron, học máy, xử lý dữ liệu và đánh giá mô hình để giải quyết bài toán thực tế. Quá trình này giúp các thành viên nắm vững quy trình thiết kế, huấn luyện, đánh giá và triển khai mô hình MLP trong một bài toán thực tế, nâng cao kỹ năng thực hành và tư duy hệ thống.
- **Cung cấp dữ liệu thực nghiệm:** Mô hình MLP được xây dựng và thử nghiệm trong môi trường mô phỏng, cung cấp dữ liệu thực nghiệm phong phú để phân tích hiệu suất, đánh giá các chỉ số như độ chính xác, ma trận nhầm lẫn và khả năng tổng quát hóa của mô hình. Đây là cơ sở quan trọng để rút ra bài học về cách thiết kế kiến trúc MLP phù hợp với dữ liệu thực tế.
- **Cơ sở để nghiên cứu nâng cao:** Kết quả học thuật từ đề tài tạo nền tảng để:
 - Nghiên cứu tối ưu MLP hoặc thử nghiệm các mô hình mạng sâu khác như **CNN, RNN, LSTM, Transformer**.
 - Mở rộng ứng dụng vào các hệ thống IoT phức tạp hơn, kết hợp nhiều loại dữ liệu (văn bản, âm thanh, hình ảnh, cảm biến).
 - Đánh giá và cải thiện các thuật toán học máy trong bối cảnh thực tế, góp phần nâng cao khả năng ứng dụng AI trong cuộc sống.

3.1.3. Hạn chế của đề tài

Mặc dù đề tài “*Phát triển hệ thống phân loại lệnh ứng dụng vào ngôi nhà thông minh*” đã đạt được nhiều kết quả tích cực, nhưng trong quá trình thực hiện vẫn tồn tại một số hạn chế cần lưu ý. Việc nhận diện và phân tích các hạn chế này giúp xác định hướng cải tiến và phát triển hệ thống trong tương lai.

Hạn chế về dữ liệu

- **Số lượng dữ liệu còn hạn chế:** Tập dữ liệu huấn luyện hiện tại chưa đủ lớn để mô hình MLP có thể học được hết các đặc trưng phức tạp của lệnh từ người dùng. Số lượng lệnh còn hạn chế dẫn đến nguy cơ quá khớp (overfitting) khi mô hình học trên dữ liệu huấn luyện mà không tổng quát hóa tốt với các lệnh mới.

- **Thiếu đa dạng và bối cảnh dữ liệu:** Dữ liệu hiện tại chưa đa dạng về ngữ cảnh, môi trường và cách diễn đạt của người dùng. Ví dụ, cùng một lệnh “bật đèn”, người dùng có thể nói hoặc nhập theo nhiều cách khác nhau. Việc dữ liệu chưa bao quát các tình huống này khiến mô hình khó nhận dạng chính xác trong môi trường thực tế, đặc biệt là các lệnh phức tạp hoặc chưa từng xuất hiện trong tập huấn luyện.
- **Chưa bao quát đầy đủ các tình huống thực tế:** Dữ liệu chủ yếu được thu thập trong môi trường mô phỏng và thiếu các tình huống thực tế như nhiễu âm thanh, trễ tín hiệu, hay các thao tác đồng thời từ nhiều thiết bị. Điều này làm giảm khả năng tổng quát hóa của mô hình khi áp dụng vào ngôi nhà thông minh thực.

Hạn chế về mô hình MLP

- **Kiến trúc MLP còn đơn giản:** MLP hiện tại vẫn ở mức cơ bản, với số lớp ẩn và neuron hạn chế. Mô hình chưa được tối ưu sâu về các tham số như số lớp ẩn, số neuron, hàm kích hoạt, tốc độ học, thuật toán tối ưu hóa, cũng như chưa áp dụng các kỹ thuật nâng cao như regularization, dropout, batch normalization để giảm overfitting và tăng khả năng học sâu.
- **Chưa so sánh với các mô hình khác:** Đề tài chưa tiến hành thử nghiệm so sánh với các mô hình học sâu khác như CNN, RNN, LSTM hay Transformer, vốn có khả năng xử lý dữ liệu chuỗi và dữ liệu phức tạp tốt hơn. Điều này hạn chế khả năng đánh giá toàn diện về hiệu năng và lựa chọn mô hình tối ưu cho bài toán phân loại lệnh phức tạp.
- **Khả năng mở rộng hạn chế:** Mặc dù mô hình có thể thêm lệnh mới, nhưng việc mở rộng số lượng lệnh lớn hoặc áp dụng cho các dữ liệu đa phương thức (âm thanh, cử chỉ, cảm biến) chưa được thử nghiệm. Điều này có thể làm giảm hiệu quả khi triển khai thực tế với nhiều loại thiết bị và tình huống khác nhau.

Hạn chế về thử nghiệm thực tế

- **Chưa triển khai trên thiết bị IoT thật:** Hệ thống mới được kiểm thử trong môi trường mô phỏng, chưa tích hợp trực tiếp với các thiết bị thực như công

tắc thông minh, đèn, quạt hay điều hòa. Do đó, các yếu tố vật lý như độ trễ truyền tín hiệu, nhiễu môi trường, hay lỗi phần cứng chưa được đánh giá.

- **Chưa đánh giá đầy đủ độ trễ và độ ổn định:** Thử nghiệm thực tế sẽ giúp đo lường thời gian phản hồi của hệ thống khi nhận lệnh và gửi tín hiệu đến thiết bị. Hiện tại, khả năng vận hành liên tục, độ ổn định trong môi trường thực và khả năng thích ứng với các tình huống bất thường chưa được kiểm chứng.
- **Khả năng thích ứng với môi trường phức tạp còn hạn chế:** Trong ngôi nhà thực, nhiều thiết bị có thể hoạt động đồng thời, môi trường có tiếng ồn, tín hiệu không ổn định. Mô hình hiện tại chưa được đánh giá trong những tình huống như vậy, nên hiệu quả thực tế có thể khác với môi trường mô phỏng.

Hạn chế về giao diện người dùng

- **Chưa xây dựng giao diện trực quan:** Hệ thống hiện tại chưa có giao diện web hoặc mobile trực quan, người dùng chưa thể nhập lệnh, theo dõi trạng thái thiết bị hay nhận phản hồi một cách dễ dàng. Thiếu giao diện trực quan làm giảm tính tiện dụng và khó đánh giá trải nghiệm người dùng thực tế.
- **Chưa tối ưu trải nghiệm người dùng:** Các tính năng quản lý, giám sát và kiểm soát thiết bị chưa được triển khai đầy đủ. Người dùng chưa có khả năng thao tác linh hoạt, chưa thể theo dõi trạng thái thiết bị hoặc nhận thông báo khi có lỗi xảy ra. Điều này hạn chế khả năng áp dụng hệ thống vào ngôi nhà thông minh thực tế.

3.2. Hướng phát triển trong tương lai

Dựa trên kết quả đạt được và các hạn chế đã phân tích, đề tài “*Phát triển hệ thống phân loại lệnh ứng dụng vào ngôi nhà thông minh*” vẫn còn nhiều tiềm năng phát triển để nâng cao hiệu quả, khả năng ứng dụng thực tế và mở rộng phạm vi nghiên cứu. Các hướng phát triển trong tương lai có thể được chia thành những nhóm chính sau:

3.2.1. Mở rộng và đa dạng hóa dữ liệu

- **Thu thập thêm dữ liệu từ nhiều người dùng và tình huống khác nhau:** Hiện tại, dữ liệu huấn luyện chủ yếu đến từ số lượng hạn chế và môi trường

mô phỏng. Việc mở rộng dữ liệu từ nhiều người dùng với các cách diễn đạt khác nhau sẽ giúp mô hình học được nhiều đặc trưng hơn, tăng khả năng tổng quát hóa khi gặp lệnh chưa từng xuất hiện.

- **Bổ sung dữ liệu đa phương thức:** Ngoài lệnh văn bản, có thể thu thập thêm dữ liệu âm thanh (giọng nói), cử chỉ tay hoặc tín hiệu từ các cảm biến môi trường. Việc này sẽ giúp mô hình có khả năng nhận dạng lệnh đa phương thức, từ đó cải thiện tính linh hoạt và khả năng áp dụng trong thực tế.
- **Xây dựng bộ dữ liệu lớn và phong phú:** Bộ dữ liệu phong phú sẽ giúp giảm hiện tượng overfitting, nâng cao độ chính xác và khả năng tổng quát hóa của mô hình. Đồng thời, dữ liệu lớn cũng hỗ trợ nghiên cứu các mô hình học sâu phức tạp hơn, từ MLP đến CNN, RNN hay Transformer, phục vụ các ứng dụng thực tế đa dạng.

3.2.2. Tối ưu mô hình MLP

- **Điều chỉnh kiến trúc và tham số:** Có thể tối ưu số lớp ẩn, số neuron trong từng lớp, hàm kích hoạt và thuật toán tối ưu hóa để cải thiện khả năng học và độ chính xác của mô hình. Việc điều chỉnh tham số cần dựa trên thử nghiệm thực tế và đánh giá hiệu suất trên nhiều tập dữ liệu khác nhau.
- **Áp dụng các kỹ thuật nâng cao:** Sử dụng **regularization**, **dropout**, **batch normalization** để hạn chế overfitting và cải thiện khả năng tổng quát hóa của mô hình. Kết hợp các kỹ thuật này sẽ giúp MLP hoạt động ổn định hơn, đặc biệt khi áp dụng trên dữ liệu phức tạp và thực tế.
- **Tăng cường học từ dữ liệu ít:** Các phương pháp như **data augmentation** hoặc **transfer learning** có thể được áp dụng để cải thiện hiệu quả học từ dữ liệu hạn chế, giúp mô hình MLP có khả năng học tốt ngay cả khi dữ liệu huấn luyện không đầy đủ hoặc bị nhiễu.

3.2.3. Nghiên cứu các mô hình nâng cao

- **Thử nghiệm các mô hình học sâu khác:** Ngoài MLP, các mô hình như **CNN**, **RNN**, **LSTM**, **Transformer** có khả năng xử lý dữ liệu chuỗi và phức tạp tốt hơn. Thử nghiệm các mô hình này sẽ giúp xác định giải pháp tối ưu cho việc nhận dạng lệnh đa dạng, phức tạp và nhanh chóng.

- **So sánh hiệu năng toàn diện:** Các mô hình cần được so sánh về độ chính xác, tốc độ xử lý, khả năng tổng quát hóa, độ ổn định để lựa chọn giải pháp phù hợp nhất với hệ thống nhà thông minh thực tế.
- **Ứng dụng cho dữ liệu đa phương thức:** Các mô hình nâng cao sẽ hỗ trợ việc kết hợp dữ liệu văn bản, giọng nói, cử chỉ và cảm biến môi trường, giúp hệ thống trở nên linh hoạt và thông minh hơn.

3.2.4. Triển khai thực tế trên thiết bị IoT

- **Tích hợp hệ thống vào thiết bị thật:** Hệ thống cần được thử nghiệm trên các thiết bị IoT thực như **ESP32**, **Raspberry Pi**, công tắc thông minh, đèn thông minh, để đánh giá hiệu suất thực tế và khả năng vận hành liên tục.
- **Kiểm thử thời gian thực và đánh giá độ ổn định:** Đo lường độ trễ, độ ổn định và khả năng thích ứng khi nhiều thiết bị hoạt động đồng thời hoặc môi trường có nhiễu. Đây là bước quan trọng để đánh giá hiệu quả và tính khả thi khi triển khai thực tế.
- **Bảo mật và phản hồi lỗi:** Cần xây dựng cơ chế phản hồi lỗi, bảo mật dữ liệu và quản lý truy cập để đảm bảo an toàn và tin cậy của hệ thống trong môi trường thực tế.

3.2.5. Phát triển giao diện người dùng

- **Xây dựng giao diện trực quan:** Giao diện web hoặc mobile giúp người dùng dễ dàng nhập lệnh, theo dõi trạng thái thiết bị, nhận phản hồi trực tiếp và kiểm soát hệ thống.
- **Hỗ trợ phản hồi thời gian thực:** Giao diện nên cung cấp thông báo tức thì về trạng thái lệnh và thiết bị, giúp người dùng giám sát hệ thống một cách thuận tiện.
- **Cải thiện trải nghiệm người dùng:** Thiết kế giao diện trực quan và dễ sử dụng sẽ nâng cao trải nghiệm, đồng thời giúp hệ thống dễ vận hành hơn cho mọi đối tượng người dùng.

3.2.6. Phát triển hệ thống đa phương thức

- **Kết hợp nhiều phương thức nhập lệnh:** Hệ thống có thể nhận lệnh từ văn bản, giọng nói, cử chỉ hoặc dữ liệu cảm biến, giúp tăng tính linh hoạt và thuận tiện trong điều kiện thực tế.
- **Tích hợp AI để nhận biết ngữ cảnh:** Việc sử dụng trí tuệ nhân tạo để nhận biết ngữ cảnh, sở thích và thói quen của người dùng sẽ giúp hệ thống phản hồi lệnh thông minh hơn, phù hợp với môi trường sống thực.
- **Mở rộng cho hệ thống phức tạp:** Hướng phát triển này cho phép hệ thống vận hành trong các ngôi nhà thông minh với nhiều thiết bị, nhiều loại lệnh, nhiều phương thức điều khiển đồng thời, đáp ứng các nhu cầu thực tế và nâng cao hiệu quả tự động hóa.

3.3. Kết luận chung

Đề tài “*Phát triển hệ thống phân loại lệnh ứng dụng vào ngôi nhà thông minh*” đã hoàn thiện một giải pháp tổng thể, tích hợp giữa trí tuệ nhân tạo và công nghệ IoT nhằm mang lại khả năng nhận dạng, phân loại và phản hồi lệnh điều khiển một cách chính xác, linh hoạt và ổn định. Qua quá trình nghiên cứu và triển khai, đề tài đã chứng minh rằng mạng nơ-ron truyền thẳng đa lớp (MLP), mặc dù là một mô hình học máy cơ bản, vẫn có khả năng xử lý dữ liệu phức tạp, học các đặc trưng quan trọng và đáp ứng các yêu cầu thực tế trong việc tự động hóa ngôi nhà thông minh.

Điều này không chỉ khẳng định giá trị ứng dụng của MLP trong lĩnh vực AI mà còn mở ra nhiều hướng phát triển cho các hệ thống tự động hóa thông minh, từ việc tối ưu hóa năng lượng, nâng cao tiện nghi, đến cải thiện trải nghiệm sống cho người dùng. Việc xây dựng một hệ thống có khả năng nhận biết và phân loại lệnh chính xác giúp các thiết bị trong ngôi nhà có thể vận hành tự động, giảm thiểu thao tác thủ công, đồng thời tạo nền tảng cho việc phát triển các hệ thống IoT phức tạp hơn, nơi nhiều thiết bị và dữ liệu đa phương thức tương tác đồng thời.

Bên cạnh giá trị kỹ thuật, đề tài còn có giá trị nghiên cứu và học thuật đáng kể. Toàn bộ quá trình xây dựng mô hình – từ thu thập và xử lý dữ liệu, thiết kế kiến trúc MLP, huấn luyện, đánh giá hiệu năng, đến mô phỏng phản hồi trên các thiết bị

– đã cung cấp một quy trình tổng thể tham chiếu cho các nghiên cứu tiếp theo trong lĩnh vực nhà thông minh và học máy. Những dữ liệu thực nghiệm và phân tích kết quả cũng là nguồn tài nguyên quan trọng để phát triển các mô hình nâng cao hơn, như CNN, RNN, LSTM hoặc Transformer, từ đó mở rộng khả năng xử lý dữ liệu đa dạng, nâng cao độ chính xác và tính ổn định trong môi trường thực.

Đề tài cũng nêu bật tầm quan trọng của việc tích hợp AI vào các ứng dụng đời sống thực. Khi các ngôi nhà thông minh trở nên phổ biến, khả năng nhận biết và phản hồi chính xác các lệnh từ người dùng sẽ quyết định mức độ tiện nghi, an toàn và hiệu quả năng lượng của toàn bộ hệ thống. Giải pháp MLP được phát triển trong đề tài không chỉ mang tính ứng dụng hiện tại mà còn tạo ra cơ sở cho các hệ thống tự học, tự tối ưu và thích ứng với thói quen người dùng, góp phần hướng tới các ngôi nhà thông minh thế hệ mới, nơi trí tuệ nhân tạo đóng vai trò trung tâm trong việc quản lý và điều khiển thiết bị.

Ngoài ra, nghiên cứu này còn mở ra nhiều cơ hội phát triển trong tương lai. Việc tích hợp đa phương thức nhận lệnh (văn bản, giọng nói, cử chỉ, dữ liệu cảm biến), nâng cao mô hình AI, tối ưu hóa bộ dữ liệu và triển khai trên thiết bị IoT thực sẽ giúp hệ thống trở nên linh hoạt, thông minh và gần gũi hơn với nhu cầu thực tế của người dùng. Các hướng phát triển này không chỉ nâng cao hiệu quả tự động hóa mà còn tạo tiền đề cho các ứng dụng AI trong quản lý năng lượng, an ninh, chăm sóc sức khỏe và các lĩnh vực khác trong đời sống thông minh.

Tóm lại, đề tài “*Phát triển hệ thống phân loại lệnh ứng dụng vào ngôi nhà thông minh*” không chỉ hoàn thành các mục tiêu nghiên cứu đã đề ra mà còn đóng góp một giải pháp toàn diện có khả năng ứng dụng thực tế cao, tạo nền tảng cho nghiên cứu và phát triển các hệ thống nhà thông minh hiện đại, linh hoạt, ổn định, thân thiện với người dùng và có khả năng thích ứng với nhiều tình huống thực tế. Qua đó, đề tài khẳng định vai trò quan trọng của AI trong việc nâng cao chất lượng cuộc sống, đồng thời mở ra các cơ hội ứng dụng rộng rãi trong tương lai, góp phần thúc đẩy sự phát triển của lĩnh vực IoT và các giải pháp nhà thông minh thông minh.

TÀI LIỆU THAM KHẢO

[1] https://vi.wikipedia.org/wiki/Trí_tuệ_nhân_tạo

[2] Nguyễn Phương Nga(ch.b) & Trần Hùng Cường, 2021, “Giáo trình trí tuệ nhân tạo Đại học Công Nghiệp Hà Nội”, Nhà xuất bản Thống kê.

[3] Nguyễn Đình Thúc, 2000, “Trí tuệ nhân tạo, mạng NơRon phương pháp và ứng dụng”, Nhà xuất bản Giáo Dục.